

Chapter 2 Building Audio Effect Plug-ins

Most of the chapters in this book present the theory and implementation of the major types of audio effects, examining the mathematical principles behind each effect. This chapter describes how to put these principles into practice by creating an audio plug-in.

Audio plug-ins are the most common way of implementing audio effects in software. A typical plug-in is a self-contained block of code which is compiled to run on a particular processor and operating system, and can be used within an audio software environment. This chapter will examine the process of creating plug-ins using the *JUCE* (*Jules' Utility Class Extensions*) programming framework, which can be used to create plug-ins for many different software platforms.

Audio plug-in basics

Audio plug-ins are usually designed for use within digital audio workstations (DAWs). To ensure compatibility across different DAWs, several industry-standard plug-in formats have been developed. These include Steinberg's *VST* (Virtual Studio Technology) format, widely supported in nearly all professional audio software; Apple's *AudioUnit* format, supported on most Mac programs; and the *AAX* (Avid Audio Extension) format by Digidesign/Avid. Each format provides similar functionality, typically including ways of passing audio into and out of the plug-in, negotiating sample rates and number of channels, and querying and setting user-adjustable parameters for the effect.

Programming language

The most common programming language for writing audio plug-ins, by far, is C++, though other languages may be used. The code will be compiled to run on a specific processor and operating system. However, the same code can typically be compiled to run on any hardware and operating system, as long as the code does not use any OS-specific functionality.

This book is not intended to cover programming fundamentals, or as an introduction to C++. Examples in the remainder of this chapter will be presented in C++ with the assumption that the reader has a basic familiarity (though not necessarily significant expertise) with the language.

Plug-in properties

The essential task of an audio plug-in is to receive an input audio signal, apply an effect to it based on some control parameters, and produce an output audio signal. In some cases, including virtual instrument or synthesizer plug-ins, no audio input is used, and the output may be produced in response to *MIDI* (Musical Instrument Digital Interface) messages. However, the effects in this book all assume an input and an output.

Audio effect plug-ins process audio. They receive digital audio and process it through to their outputs. Their function is similar to that of hardware audio processors. They can also be chained together. That is, most hosts will allow the audio output from one effect to be used as input to another effect.

In contrast, virtual instrument plugins generate audio. They can act as standalone software synthesizers, samplers, or digital musical instruments. They typically use MIDI messages to control instrument parameters. Some effect plug-ins also accept MIDI input. In fact, the use of MIDI for control is not the distinguishing factor between effects and instruments. Think of the instruments as being similar to the effects, but without the need for digital audio input.

There are also other types of audio plug-ins. Monitoring effects are a special type of effect that provides feedback about an input signal without processing audio. Typically, they present a visualisation of the input signal, like a level meter. Think of these as having digital audio input, but no need for any output signal.

Finally, it's worth mentioning MIDI effects, which do not really operate on audio at all. They have MIDI input and MIDI output. So they could, for instance, apply pitch shift on a MIDI data stream. The output MIDI messages could then be sent to other virtual instruments or hardware devices.

Our focus is on just the audio effect plug-ins. Important properties of such a plug-in include the *number of channels* it supports and the allowable *sample rates*. Many effects can operate with different numbers of channels, but others will require specific channel configurations. For example, a stereo panning effect would need at least two output channels, but it could take one or two input channels for stereo or mono input). Some effects may have restrictions on the sample rates they support, though it is useful wherever possible to write plug-ins that operate at any sample rate. Plug-ins will also define one or more user-adjustable *parameters* which can typically be changed either through a standard interface provided by the DAW, or by a custom GUI created by the plug-in author.

The plug-in host

A plug-in host is a software application or hardware device in which the plugins run. It typically presents plugin user interfaces and routes digital audio and MIDI to and from plugins.

The most well-known form of host for audio effect plug-ins is the digital audio workstation, or DAW. These are applications and devices used for a wide range of audio production tasks, such as recording, editing and rendering multitrack music or soundtracks. However, audio effect plug-ins might also be used in a standalone host which lacks some DAW features but has been optimized for use in live performance. They might also be used in video editing tools, or in game engines.

The JUCE framework

The example code for this book uses the *JUCE* (Jules' Utility Class Extensions) environment, created by Julian Storer, which provides a cross-platform, multi-format method for building audio plug-ins. JUCE was created in 2004 by Jules Storer. It is an open source cross-platform C++ codebase for developing desktop and mobile applications, and plug-ins. It is used to write software so that it will run with the same user experience on many platforms (Windows, macOS, Linux, iOS, Android...) and in many formats. JUCE

furthermore provides a cross-platform set of graphical user interface (GUI) controls and a very large library of useful C++ classes covering commonly used features by apps and plug-ins, such as graphics, audio, XML parsing, networking, cryptography, multi-threading... Thus, it also reduces the number of third-party libraries needed in a project.

But the real reason why we focus on JUCE here is that it is the *most widely used framework for audio application and plug-in development*. As mentioned, JUCE has a very large number of built-in libraries, but this is especially true for those related to audio functionality. JUCE has support for audio devices (CoreAudio, JACK, DirectSound) and MIDI playback, DSP building blocks, polyphonic synthesizers..., and built-in readers for common audio file formats (WAV, AIFF, FLAC, MP3, Vorbis...). It also comes with wrapper classes for building all major audio plugin formats. Since all platform and format-specific code is contained in the wrapper, one can build for almost any plug-in format & platform from a single codebase.

JUCE is free for use in open-source projects. Documentation and download links can be found on its website: <http://www.juce.com>.

VST – Virtual Studio Technology

The most popular audio plugin format is VST, which stands for Virtual Studio Technology. VST was created by Steinberg (now owned by Yamaha) in 1996. VST is an audio plug-in software interface. That is, it is an audio plugin standard that allows virtual instruments and effects (and more, as we will see) to be integrated into digital audio workstations. It is the main industry standard for audio plug-ins.

VST plug-ins usually run within digital audio workstation (DAW), though stand-alone VST plug-in hosts also exist. VST plug-ins are intended to provide additional functionality to the host, and they often use digital signal processing to simulate traditional recording studio hardware. VSTs usually have graphical user interfaces that display controls similar to physical switches and knobs on audio hardware.

Most VSTs are either instruments (VSTi) or effects (VSTfx), though other categories exist, such as the monitoring effects mentioned previously.

Theory of operation

JUCE audio plug-ins are divided into two components: a *processor* that handles the audio calculations and an *editor* or GUI that lets the user interact with the plug-in. The processor provides several functions: a *callback function* that the DAW calls every time it needs a new block of audio samples; methods for getting and setting effect parameters; and initialization and cleanup routines. The editor provides graphical controls for the user to see and change the parameters. Most DAWs will provide a generic editor when the plug-in does not define its own.

Callback function

The most important task for an audio plug-in is receiving and processing audio samples. How does the plug-in know how many samples to process, and when to process them? If the effect is operating in real time, it is clearly impossible to wait for the entire audio signal to arrive before applying the effect. Instead, audio needs

to be processed in small *blocks* or *buffers* of samples as it comes in. To receive blocks of samples, plug-ins implement a *callback function*, a function which the DAW calls every time it has new audio to process. Thus, it is always the DAW, and *not* the plug-in, which determines how many audio samples to process and when. The advantage of this arrangement is that the plug-in author never needs to be concerned with where the audio samples come from, when they should arrive, or where they go after the effect has been applied. The author simply needs to write a callback function which processes as many samples as requested by the DAW.

When the DAW runs the callback function, it will provide several pieces of information. These include the *buffer size* (how many audio samples to process), the *sample rate*, the *number of input and output channels*, and a *buffer* (region of memory) containing the input audio. The host will also provide a buffer in which the audio output should be stored. In JUCE (as in many plug-in formats), the plug-in is expected to put its output in the same buffer where the input samples were found.

When the callback function finishes, it returns control to the DAW, which decides what should happen to the processed samples. The callback function will be called again when there are more samples to process. The buffer size used by the DAW partly determines the overall *latency* (delay) from input to output; smaller buffer sizes produce a low delay but increase the risk of *underruns* (gaps) if the computer cannot respond quickly enough. A typical conservative buffer size would be 512 samples; a high-performance buffer size might be as small as 32 samples. At a 44.1kHz sample rate, a buffer size of 32 means that the callback function would run over 1300 times per second!

Managing parameters

Nearly every audio effect will have one or more user-adjustable parameters. For example, a parametric equalizer plug-in might let the user change the center frequency, the gain and the Q of the filter. Most plug-ins provide a user interface to allow the user to adjust parameters. Every operating system and plug-in format provides different routines for managing a GUI, but there are several common requirements. In particular, the plug-in must provide a set of functions for the user to see or change the parameters, and there must be a way for the callback function to discover the current parameter values.

JUCE provides a convenient class for almost any parameter that the plug-in would expose to the DAW, the `audioProcessorParameter`. JUCE also provides several *methods* (C++ object functions) related to managing parameters, described in detail in the example in the next section. Two of the important methods are `getParameterTree()` and `setParameterTree()` within the audio processor object (see Figure 2.1). When the DAW calls `getParameterTree()`, the group of parameters managed by that audio processor is returned. Parameter values are most often in the form of floating-point numbers (C++ type `float`). It is up to the plug-in to define what they mean (for example, a parametric equalizer might have frequency as parameter 0, gain as parameter 1 and Q as parameter 2). With `setParameterTree()`, the group of parameters managed by the audio processor can be set. The host provides an index of the parameter to set and a floating-point value it should be set to. The p11 plug-in stores this information so the audio callback

function can access it later. In particular, the callback function, `processBlock()` accesses the values of these variables to discover the current parameter settings.

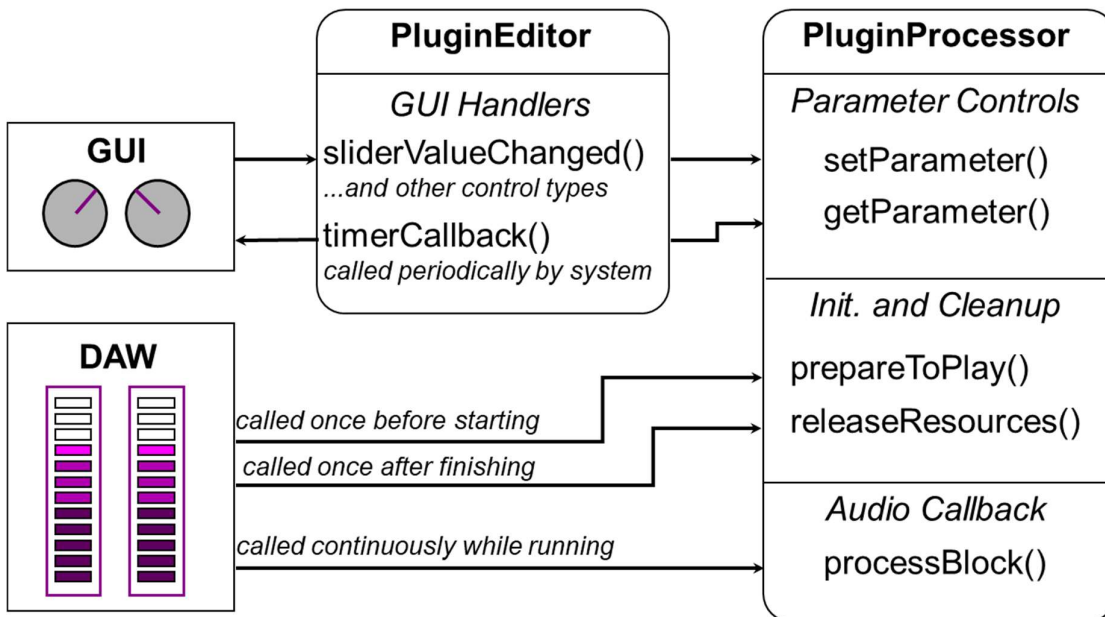


Figure 2.1. Basic components of a JUCE audio plugin and their relationship to the digital audio workstation (DAW).

Initialization and cleanup

Before a plug-in can process audio, certain initialization tasks must be performed. For example, in a delay plug-in, memory for delay buffers might need to be allocated; for an equalizer plug-in, filter coefficients might need to be calculated and internal variables holding previous samples might need to be initialized to 0. For plug-ins with low-frequency oscillators, the phase of the oscillator may need to be initialized.

Every plug-in format will provide a method for initializing the plug-in. In JUCE, basic initialization can be performed in the *constructor* of the audio processor object, which runs once when the plug-in is first loaded. JUCE also provides a method called `prepareToPlay()` which runs immediately before audio processing begins. By allocating resources just before the audio starts, the plug-in only uses resources while actively running.

All resources that are allocated by the plug-in will eventually need to be released. JUCE provides a method called `releaseResources()` which runs immediately after the host stops processing audio. This method should be used to free any resources allocated in `prepareToPlay()`. It is safe to assume that the audio callback function will never run after a call to `releaseResources()`. Similarly, the counterpart to the C++ constructor is the *destructor*, which runs once when the user removes the plug-in from the host environment. Anything that is allocated in the constructor should be freed in the destructor.

Preserving state

Each time the DAW calls the plug-in's callback function, it will provide only a small block of input samples to be processed. The plug-in often depends on previous state information to know how to process these samples. For example, phases of oscillators, pointers within circular buffers, and previous values of input and output samples may be needed to calculate the output. It is up to the plug-in to save any state that it needs during the callback function.

To understand the importance of managing state, consider a simple effect where the current output is equal to the sum of the last two inputs: $y[n] = x[n] + x[n-1]$. Suppose that the host requests 512 samples beginning at sample N . It will therefore supply a buffer containing 512 input samples $x[N]$ to $x[N+511]$ and require 512 output samples $y[N]$ to $y[N+511]$.

At first glance, it may seem as if the callback function has enough information to calculate the output without any reference to what has happened before. But what about calculating $y[N]$, the very first sample in the buffer? We have $y[N] = x[N] + x[N-1]$, but $x[N-1]$ was supplied *last time* the callback was run, and that value is no longer available. The plug-in must therefore use a separate instance variable to remember what has come before. Here, a single `float` for each channel would be needed to remember the previous sample. The float should be declared inside the class, so its value is preserved across calls to the callback function.

Set up

This section briefly covers how to set up the tools to begin audio plug-in development.

Required software

To build the plug-ins that come with the book, the following software is required:

1. The *JUCE (Jules' Utility Class Extensions)* C++ library by Julian Storer. JUCE runs on nearly every platform, including Mac, Windows and Linux, and can be downloaded free from <http://www.juce.com>.
2. An Integrated Development Environment, or IDE, with a C++ compiler. This is an application with lots of tools for software development, like a code editor and a debugger. Common ones include Visual Studio for Windows, Xcode for MacOS and iOS, Android Studio, and Code::Blocks. They are all generally available as free downloads, sometimes after a registration process. Here, we will do everything in Windows with Visual Studio, but the steps should be similar with different development environments and operating systems.
3. To build VST plug-ins, the VST3 SDK (2.x versions are still supported but no longer recommended) is required. This is bundled with JUCE. Optionally on Mac, AudioUnits can be created instead. Whether you use VST or AudioUnits, your program in JUCE will be written identically.
4. A suitable digital audio workstation (DAW) or other host environment in which to test the plug-ins. JUCE ships with a VST plug-in host, which is sufficient for the sort of testing we need. On the Mac, the Xcode Audio Tools provide the *AU Lab* program which is a simple, lightweight way of testing AudioUnit plug-ins. Several other cross-platform options are available, including Audacity and Reaper.

Download the tools

If you do not have one already installed, you will need to download an IDE. When installing, you should be able to do a basic installation, needing only C++.

Now download JUCE from <https://juce.com/download/> . Unpack the JUCE folder and put it in some convenient location on your computer.

Setting up the Projucer

In JUCE, new projects are configured using a program called the *Projucer*. The Projucer is JUCE's tool for creating and managing JUCE projects. Once files and settings for a JUCE project are specified, it automatically generates 3rd-party project files to allow project to compile natively on each target platform; Visual Studio, Xcode, Android Studio, CodeBlocks, Linux Makefiles... The Projucer also has a code editor, integrated GUI editor, and wizards for creating new projects and files.

Depending where you download JUCE from, it may or may not come with a pre-built version of the Projucer. If not, then the Projucer needs to be compiled and built on your system before anything further can take place. Within the main JUCE folder, Projucer project files for each of the major development environments (Visual Studio, Xcode, etc.) can be found in [extras/Projucer/Builds](#). Open the project in your development environment and build it. It should compile without errors if JUCE was correctly installed. Run the Projucer once it finishes compiling.

A common problem is that the JUCE libraries, or modules, are not easily found. The steps to set their location are shown in Figure 2.2, and are as follows. First, the Global paths may need to be set in the Projucer app. Navigate to the menu item *File -> Global Paths* on Windows or Linux, or *Projucer -> Global Paths* on MacOS. Open global paths and set them to the right locations (1), if they are not set there by default. Now close the global paths window. Then, click in the *Modules* section on the left (2). Click on the cog wheel and scroll down (3). Select *Enable/disable global path for modules...* and ensure that the Global Path is used for all modules (4).

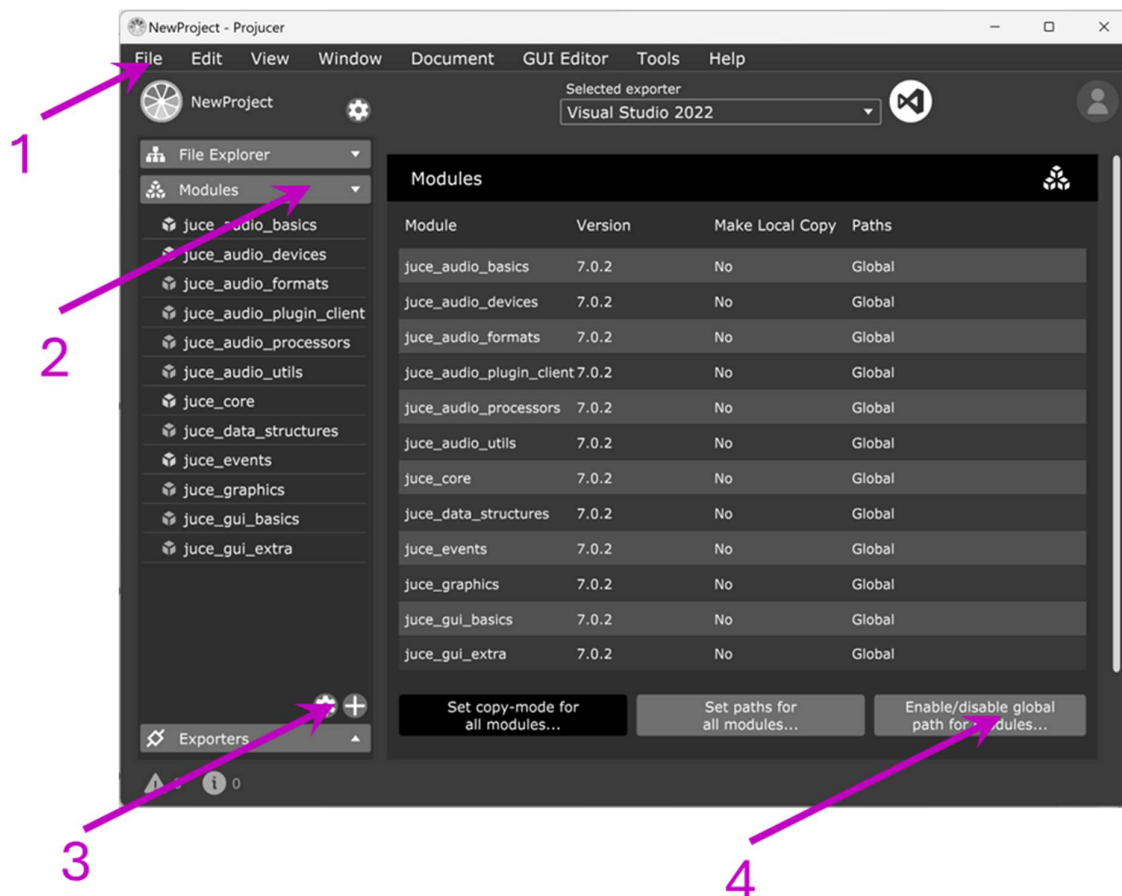


Figure 2.2. The steps in the Projucer for setting the global path for modules.

The Projucer can now be used to create new JUCE projects, view tutorials, run examples and much more. It is also possible to include the JUCE modules source code in an existing project directly, or build them into a static or dynamic library which can be linked into a project.

Opening example plug-ins

This book comes with several example audio plug-ins which can be opened in the IDE or your choice. Within each plug-in directory, look in the `Builds` directory for a version specific to your IDE. If you do not find one, open the `.juicer` file in the Projucer and create a new target for your platform. Following the instructions in this chapter, you should be able to compile the plug-in and get it running in your audio host environment.

Example: building a gain effect in JUCE

This section will describe the implementation of a simple plug-in in the JUCE environment. We will take as an example a basic gain effect, with user-adjustable control for a gain parameter that multiplies the incoming signal to produce the outgoing signal. It will demonstrate many of the concepts that have been discussed, including the management of parameters and preservation of state between callbacks.

The complete code for this effect can be found in the materials that accompany the book, along with example code for several other effects.

Creating a new plug-in in JUCE

Once the Projucer is running, select *New Project* from the *File* menu, which brings up the *New Project* window. The Projucer will automatically generate different starter files depending on what you want to develop. Enter a project name (here, we call our project HelloWorld) and set the Project Type to Plug-In ->Basic (see Figure 2.3).

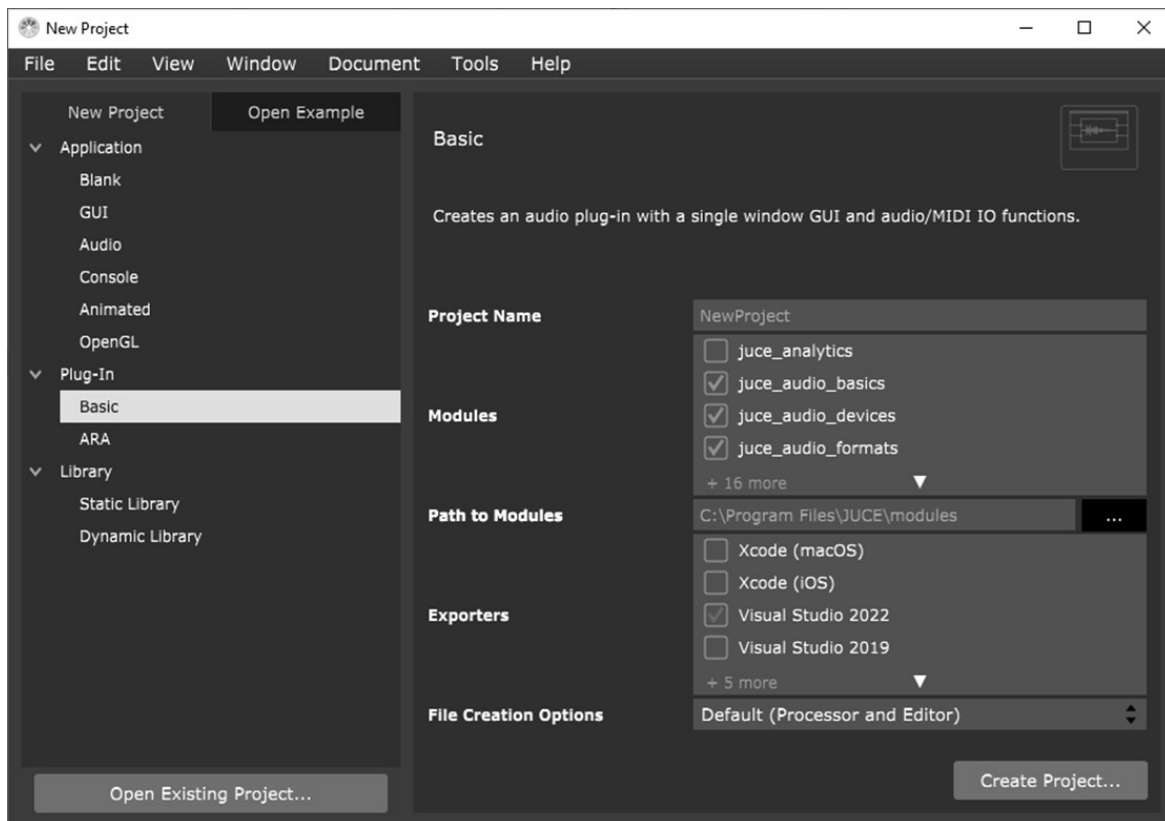


Figure 2.3. The new project window, for creating a new audio plug-in with the Projucer.

Click Create Project to make the JUCE project for your new plug-in. The Projucer displays a new screen now. On the left there is an accordion (stacked list user interface element), with three headings,

- File Explorer
- Modules
- Exporters

The next step is to make an export target for your specific Integrated Development Environment. By default, the Projucer should create a target for the IDE on your platform. If not, or if you want to make an additional target, right-click within the Exporters section on the left, or click on the plus symbol in that section, to create a new export target (Figure 2.4).

Selecting the export target from the list on the left-hand side of the window brings up settings specific to the platform. Most of these options can be left at their default values. Similarly, module settings can also usually be left at their defaults.

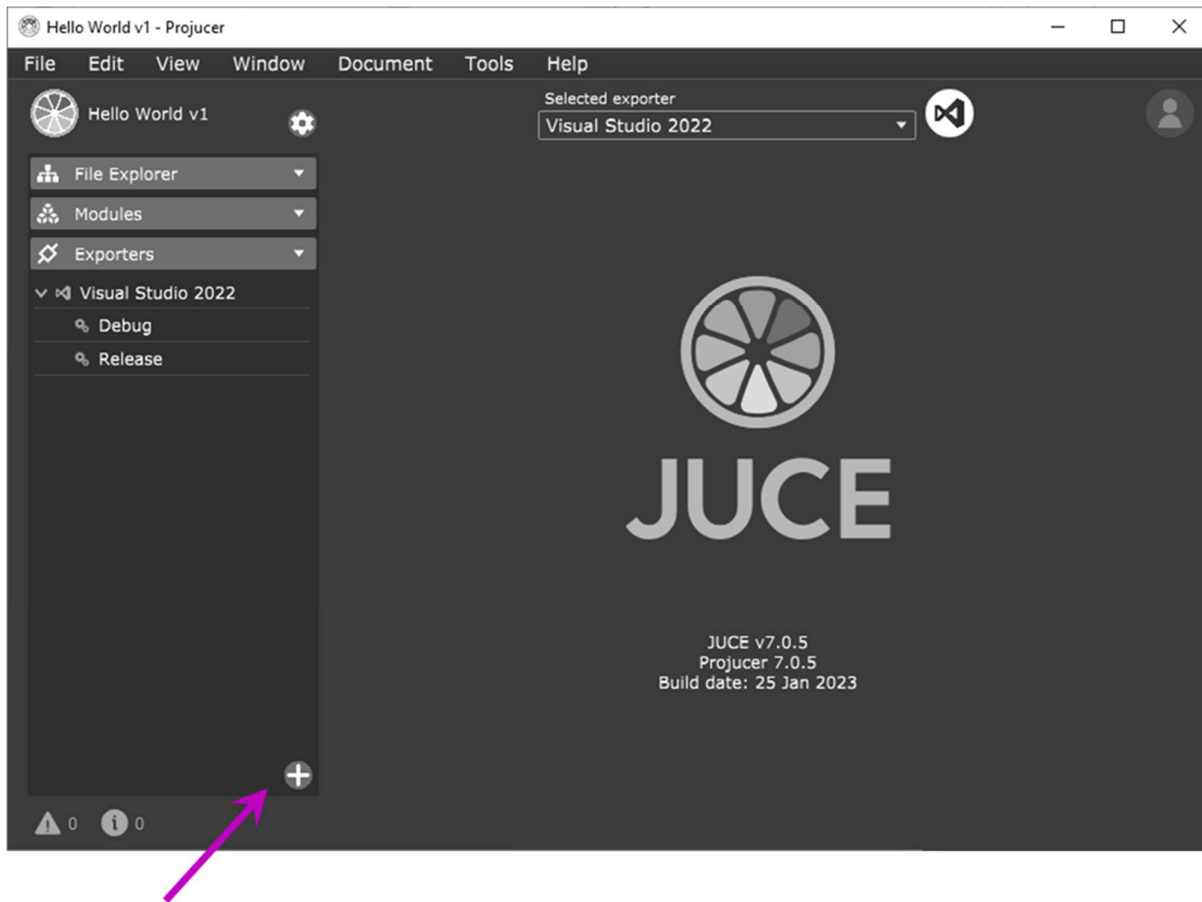


Figure 2.4. Adding an export target in the Project window.

Now click on the cog wheel near the top left corner to get the *Project Settings* window where you can edit the details of the plug-in (Figure 2.5).

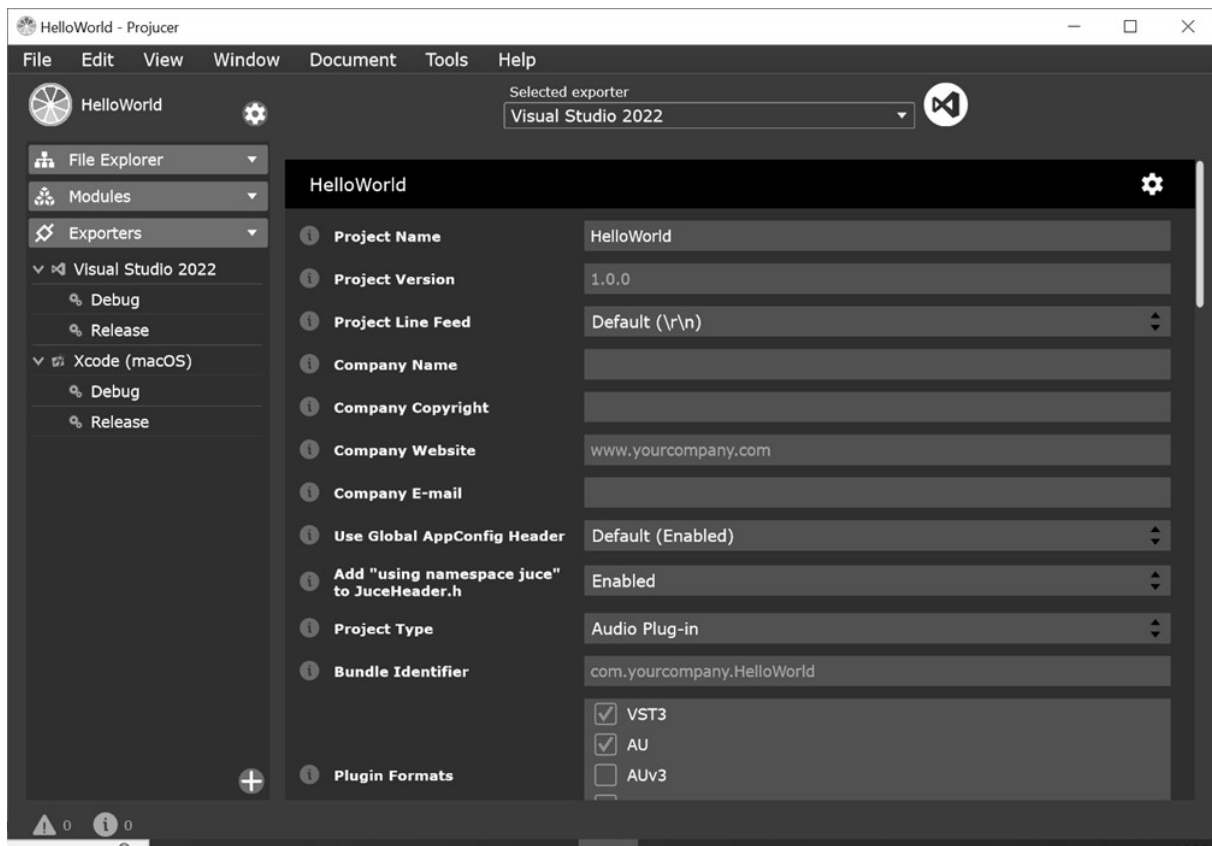


Figure 2.5. The Project Settings window.

There are several fields worth mentioning here;

- *Project Name*: the name of the project. This is used for naming various methods and files when the Projucer generates code. They can be renamed, but its useful to pick a preferred name at this stage.
- *Project Version*: the version number of your plug-in (the default value is fine for new plug-ins)
- *Company Name*: the name of your company or organization, as you want it to appear in the audio host environment
- *Bundle Identifier*: a unique identifier for the plug-in. The format looks like a reversed web address, which identifies the organization and the plug-in name.
- *Plugin formats*: these boxes select which format of plug-in to build. On most platforms, including Windows, VST3 is the most common format; on Mac, AudioUnit is the usual selection. Selecting those two checkboxes is a common option.
- *Plugin Name*: the name of your plug-in as you want it to appear in the host environment, usually the same as the Project Name
- *Plugin Description*: a short description of your plug-in
- *Plugin Manufacturer*: company name, similar to *Company Name* above
- *Plugin Manufacturer Code*: a four-letter code identifying your company or organization. Use a single code consistently across all the plug-ins you develop.

- *Plugin Channel Configurations*: pairs of numbers which identify valid configurations of input channels and output channels. For example, `{1,1}`, `{2,2}` indicates a plug-in that can take mono input and mono output, or stereo input and stereo output. The default is to leave this field blank.

Edit these settings as you wish, but make sure to give the plug-in an appropriate name and check VST3 and/or AU for the plug-in format. The other settings can usually be left at their defaults.

When you have finished entering all the details, click the icon next to *Selected exporter* at the top of the window. This should bring up the project within your IDE, where it can be built using the same procedure you normally use to compile projects. On the Mac, building the project will automatically copy the resulting plug-in to the system plug-ins directory (`~/Library/Audio/Plug-Ins/Components/`) where it will be found by audio software including AU Lab. On Windows, your DAW may need to be manually set to look for your plug-in. In many cases, restarting the audio host program (AU Lab, Audacity, etc.) is necessary every time the plug-in is recompiled.

File overview

Juce plug-ins contain four source files by default:

- *PluginProcessor.h*: the header file for `PluginProcessor.cpp`; defines the C++ object whose code is filled out in `PluginProcessor.cpp`. Includes declaration of methods and object variables.
- *PluginProcessor.cpp*: the main file where the audio processing is done.
- *PluginEditor.h*: the header file for `PluginEditor.cpp`, containing the definition of the C++ object for the editor.
- *PluginEditor.cpp*: contains the code for the graphical user interface for the plug-in.

In your own projects, for more complex effects, you might find occasion to add extra source files to the project. But most effects can be implemented in just these four. We will examine each file in turn. Not all lines of each file are included in this text, so please refer to the accompanying materials for the complete source.

These files contain a lot of auto-generated code, but you don't need to do anything about most of this code yet. To start we will focus on the `PluginProcessor.cpp` file, so open up this file in your IDE. This is where most of the audio-related code is located.

Scroll down until you reach the method called `processBlock()`.

When JUCE auto-generated the project files, a buffer array was created. The `processBlock()` method takes an argument of type `juce::AudioBuffer<float>`. This is a multidimensional array of samples that changes depending on the number of channels and the block size set in your DAW. So if your block size is set to 512 samples and your plugin processes two channels of audio, there will be two arrays of length 512. We will use a `for` loop to change the data in this array, by iterating through the array for each channel, and changing the value of the samples.

Now edit the code at the end of `processBlock()` so it looks like this:

```
for (int channel = 0; channel < getTotalNumOutputChannels(); ++channel)
{
    auto* channelData = buffer.getWritePointer(channel);
    for (int sample = 0; sample < buffer.getNumSamples(); sample++)
    {
        channelData[sample] = channelData[sample];
    }
}
```

This loops through channels of input data. There could be any number of channels, but the most common situation is dealing with stereo audio, where channel 0 is the left channel and channel 1 is the right channel. Currently, for every channel, this iterates through the audio buffer but does not change anything. The plug-in should compile and build successfully, but passing an audio signal as input to the plug-in will produce the same signal at the output.

This general structure is the starting point for lots of different audio effect plug-ins.

Now let's use the inner `for` loop to change the value of each sample in the array.

Let's add a floating point gain variable inside `processBlock()` and set its value to something less than 1.

We will then multiply every sample by this gain.

```
float gainValue = 0.1f;
for (int channel = 0; channel < getTotalNumOutputChannels(); ++channel)
{
    auto* channelData = buffer.getWritePointer(channel);
    for (int sample = 0; sample < buffer.getNumSamples(); sample++)
    {
        channelData[sample] = gainValue * channelData[sample];
    }
}
```

First, this assigns a variable `channelData` as the write pointer of the audio buffer. It then multiplies the input by a gain equal to 0.1. The input is written back into channel data. The `for` loop then increments the input to the next sample, and repeats.

So this code multiplies every sample value in the audio buffer by a gain of 0.1. When this plugin is placed on a track in your DAW it will reduce the level of the incoming signal.

Let's try this out now in your DAW, first build the project. This will create a VST3 file in the project directory usually in a location like `Builds->VisualStudioXX->x64->Debug->VST3`. Open your DAW of choice and set this folder as a VST location, or move the built file to your VST directory. You should hear that it has applied a gain reduction to any audio used as input to the plug-in.

However, the user has no control over this gain reduction. Let's give the plug-in a user interface.

The `GenericAudioProcessorEditor` is a user interface component that displays the parameters of an `AudioProcessor` as a basic list of sliders, combo boxes and switches. It will also automatically lay out the

controls on an interface. Here, we will use it to greatly simplify the user interface design and control, allowing us to focus mainly on the audio-related aspects.

For our plugin to use this generic interface, we replace the line

```
return new HelloWorldAudioProcessorEditor (*this);
```

in the PluginProcessor.cpp file with

```
return new juce::GenericAudioProcessorEditor(this);
```

We then open the PluginProcessor.h file, and add this variable to the private section;

```
juce::AudioParameterFloat* gainParam;
```

In the constructor in PluginProcessor.cpp, add a parameter to the audio processor.

```
addParameter(gainParam = new juce::AudioParameterFloat("gain", "Gain", 0.0f, 1.0f, 0.0f));
```

where

- *gainParam* references the AudioParameterFloat we created in PluginProcessor.h.
- *gain* is the slider's ID
- *Gain* will be shown next to the slider in the DAW.
- Float values set minimum, maximum, and default values for the slider

Since we are using the generic interface, it will automatically add a slider to the user interface, which can be used to control this audio parameter. However, the slider doesn't yet do anything. So let's modify the code in the processBlock to be

```
float gainValue = gainParam->get();
```

Build and run it again. Now we have user control of the gain slider, as shown in Figure 2.6, so that the user can change the volume of incoming audio data.

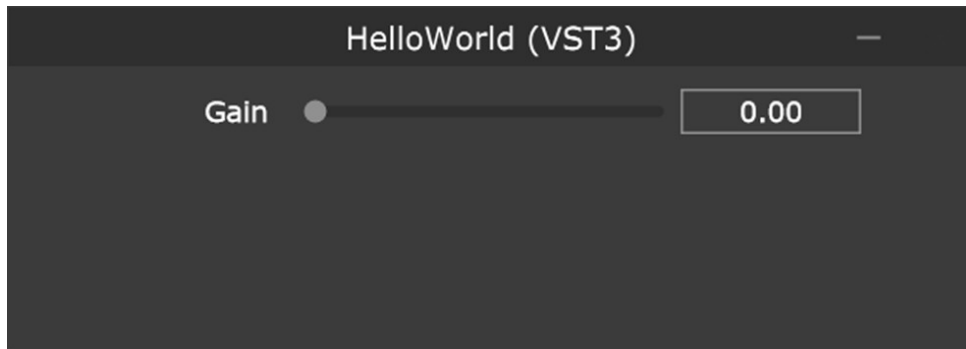


Figure 2.6. The interface of the HelloWorld plug-in.

Add the new VST effect to an audio channel and you should see the following results:

Here, we created a basic gain plugin. This isn't very exciting, but it shows the process of developing and testing audio plugins using JUCE. More complicated plugins are made mainly by modifying the process block and adding more UI elements to further control the effect of these modifications.

Summary

This section has examined the code for an example plug-in that implements a simple gain control. Four source files were required, two related to the audio processor and two related to the user interface. The most

important task for the audio processor is to implement the *audio callback* function `processBlock()`, which the system calls every time it needs a new block of audio.

Since audio is processed in small blocks, it is necessary to remember the state of the effect between callbacks. So for more complex effects than the simple gain example given here, *instance variables* should be declared in `PluginProcessor.h` to record information that persists beyond the duration of `processBlock()`.

The other important task was the management of effect parameters. Ordinarily, this would require careful coordination between all four source files; parameters to be declared in `PluginProcessor.h` alongside instance variables, controls to change their values declared in `PluginEditor.h`, and methods in both `PluginProcessor.cpp` and `PluginEditor.cpp` for getting and setting parameters. However, with the use of the generic user interface, the editor files are not needed at all. We can focus just on the audio-related content in the plugin processor files.

Conclusion

This chapter has shown how to put the principles of building audio effects into practice through the creation of audio plug-ins. The JUCE framework is a convenient environment for creating audio plug-ins since it allows the same code to be compiled into multiple plug-in formats on multiple operating systems. However, the general principles of writing audio callback functions, managing parameters, allocating resources and creating a user interface are similar across many plug-in formats. The example code included with the book offers templates from which you can start building your own effects, including examples from most chapters and a blank plug-in into which you can add your own code.

Further reading

Excellent introductions to C++ programming can be found in many sources, including [5, 6, 7]. An audio-focused introduction to C++ is included in [8], which also goes into detail on many aspects of audio programming not covered in this text.

There are a large number of texts dedicated to audio programming, each one focusing on different aspects. For working with JUCE, the ‘JUCE framework tutorials’ videos¹ by the Audio Programmer are especially helpful and popular. And somewhat complementary to aspects of this text, are the two texts by Will Pirkle on designing audio effects [9] and virtual instruments [10]. However, machine learning is generally outside the scope of both this text and those resources. For development of audio plugins that use AI or machine learning, the reader is recommended to consult [11]. Finally, recent useful resources are available for programming with many audio-specific development environments and tools, such as PureData [12] and the Web Audio API [13].

¹ <https://www.youtube.com/playlist?list=PLlgJJsrDwhPxa6-02-CeHW8ocwSwl2jnu>